

■ EasyService — Comprehensive Testing Guide & Path

Course: Software Testing & Quality Assurance (Testing & Validation)

Project: EasyService Ethiopian Marketplace Platform

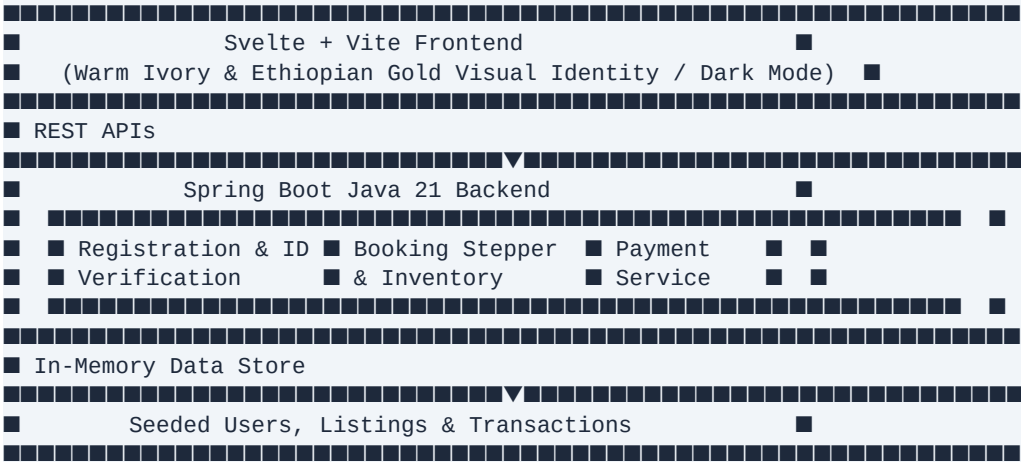
Authors: Abel Seleshe (ATE/6743/14), Baheran Tesfaye (ATE/5750/14), Nebiyu Yohannes (ATE/3973/14), Wondesen Teshale (ATE/4671/14)

Date: September 2, 2026

■ 1. What is EasyService and How It Works

EasyService is a modern, high-fidelity Ethiopian service and booking marketplace. It connects customers (Ethiopian citizens and international visitors) with local service providers offering **Hotels & Resorts, Car Rentals, Cultural Events, and Local Artisan Products**.

Core Architecture & Flow:



■ 2. System Credentials & Persona Matrix

Role	Name	Email / Phone	Password	Customer Type	Identity Type	ID Number	Initial Balance
Ethiopian Citizen	Abebe Kebede	`user1@aaau.edu.et` `+251911000001`	`Password123` `!`	`ETHIOPIAN`	`FAYDA`	`FY10001` `	ETB 5,500.00
Foreign Visitor	John Foreigner	`john@global.com` `+12025550199`	`Password123` `!`	`FOREIGNER`	`PASSPORT`	`P90001`	ETB 8,000.00
System Administrator	Platform Admin	`admin@easyservice.com` `+251911999999`	`AdminPass123` `!`	`ETHIOPIAN`	`FAYDA`	`FY000000`	ETB 100,000.00

Business Provider Accounts:

- **prov_1**: Kuriftu & Bishoftu Lake Resorts Provider
 - **prov_2**: Safari & Land Cruiser Car Rental Provider
 - **prov_3**: Great Ethiopian Run & Ethio-Jazz Event Organizer
 - **prov_4**: Shiro Meda Handcrafted Ethiopian Goods Artisan
-

■ ■ 3. Application Services & Business Rules

1. **RegistrationService**: Handles user onboarding. Enforces email regex, phone formatting, unique credentials, and identity verification checks.
 2. **IdentityVerificationService**: Enforces **BR-02** — Ethiopian citizens must use valid **Fayda ID** (**FY...**); Foreign visitors must use valid **Passport** (**P...**).
 3. **ListingService**: Manages service inventory (**Hotels, Cars, Events, Products**). Enforces positive pricing (**BR-05**) and positive capacity (**BR-06**).
 4. **BookingTransactionService**: Manages 4-step booking workflow, state transitions (**PENDING** -> **CONFIRMED** -> **COMPLETED** / **CANCELLED** / **EXPIRED**), inventory locking (**BR-15**), and cancellation refunds (**BR-14**).
 5. **PaymentService**: Simulates wallet balance validation. Enforces **BR-08** — declines transactions if wallet balance is less than required total.
 6. **PromotionService**: Calculates promotional discounts for valid codes (**SUMMER20**, **GOLD15**, **ETHIO30**, **AAU10**).
 7. **EasyToolsService**: Provides persona quick-switching and custom wallet top-up logic for testing.
-

■ 4. How to Test Everything

A. Testing the Frontend UI & Web Application

1. Open terminal in **frontend/**:

```
cd "c:\Users\25194\Desktop\Testing and Quality Assurance\GROUP PROJECT\Easyservice\frontend"
npm run dev
```

2. Open browser at **http://localhost:5173**.

3. Test key interactions:

- **Theme Switcher**: Click ■ ■ **Light** / ■ **Dark** in top navbar.
 - **Spin the Wheel**: Click ■ **Spin Wheel** to get a discount code (e.g. **SUMMER20**).
 - **Wallet Top Up**: Enter any amount (e.g., **5000**) in the wallet tool and click + **Add**.
 - **Booking Flow**: Click **Book Now** on any listing -> Select dates -> Apply promo code -> Confirm simulated payment.
 - **My Bookings & Refund**: Open **My Bookings** -> Click **Cancel** -> Verify booking becomes **CANCELLED** and funds refund to wallet.
-

B. Running Automated Unit & Integration Tests (Maven)

1. Open terminal in `backend/`:

```
cd "c:\Users\25194\Desktop\Testing and Quality Assurance\GROUP PROJECT\Easyservice\backend"
mvn test
```

2. What gets executed:

- `AuthServiceTest.java` (Authentication & token generation)
 - `RegistrationServiceTest.java` (Input validation & duplicate handling)
 - `IdentityVerificationServiceTest.java` (Fayda vs Passport decision matrix)
 - `ListingServiceTest.java` (Listing creation & capacity controls)
 - `BookingTransactionServiceTest.java` (State machine transitions & refunds)
 - `PromotionServiceTest.java` (Discount calculations)
 - `EasyToolsServiceTest.java` (Wallet balance deposits)
-

C. Running the 100 Marketplace Break Test Suite

To run the automated destructive edge-case tests:

```
mvn test -Dtest=EasyServiceBreakTestSuiteTest
```

This tests:

- Multithreaded inventory race condition protection (`testConcurrencyInventoryRaceCondition`)
 - Registration missing fields & weak passwords (`testRegistrationBoundaries`)
 - Inverted price range filters (`testPriceRangeFilterInversion`)
 - Wallet balance decline (`testInsufficientBalancePaymentDecline`)
 - Double cancellation refund idempotency (`testCancellationDoubleRefundPrevention`)
-

D. Running Automated Selenium E2E Tests

The platform includes automated browser testing powered by **Selenium WebDriver** and the **Page Object Model (POM)** pattern.

Test Execution:

```
cd "c:\Users\25194\Desktop\Testing and Quality Assurance\GROUP PROJECT\Easyservice\backend"
mvn test -Dtest=EasyServiceSeleniumE2ETest
```

Selenium Architecture & Page Objects:

- **Test Class:** `com.easyservice.backend.selenium.EasyServiceSeleniumE2ETest`
- **Page Objects:**
- `ListingPageObject.java`: Locates marketplace cards, filter controls, search bar, and category buttons.

- **BookingPageObject.java**: Locates modal inputs, date pickers, promo input, payment buttons, and receipt confirmation.

- **Selenium Scenarios Tested:**

1. Opens Chrome browser and navigates to **http://localhost:5173**.
 2. Filters catalog by location (**Bishoftu**) and category (**HOTEL**).
 3. Opens Kuriftu Resort listing and clicks **Book Now**.
 4. Selects dates, enters **SUMMER20** promo code, and verifies price calculation.
 5. Clicks **Confirm Payment** and asserts receipt modal with Transaction ID.
 6. Navigates to **My Bookings** and asserts booking status is **CONFIRMED**.
-

■ 5. How to Open and View All Documentation Files

All project documentation is located in the **docs/** folder:

File Name	Description	How to Open
`Testing_Guide_Path.md`	Master Testing & Execution Guide	Open in VS Code, Notepad, or Markdown Viewer
`EasyService_100_Marketplace_Break_Test_Specification.md`	100 Break Test Plan & Specification	Open in VS Code or Markdown Viewer
`EasyService_Master_Specification_v1_0.md`	Architecture & Business Rules Spec	Open in VS Code or Markdown Viewer
`Test_Plan_Part_A.md`	QA Test Strategy & Environment Setup	Open in VS Code or Markdown Viewer
`Test_Design_Part_B.md`	Test Cases & Decision Tables	Open in VS Code or Markdown Viewer
`Defect_Log_And_Metrics_Part_FG.md`	Defect Tracking & Quality Metrics	Open in VS Code or Markdown Viewer
`Test_Summary_Report_Part_HI.md`	Final Test Execution Summary	Open in VS Code or Markdown Viewer
`Microservices_Technical_Report_30_50_Pages.md`	Full Technical Cloud Report	Open in VS Code or Word
`Microservices_Presentation_20_Slides.md`	20-Slide Presentation Outline	Open in VS Code or PowerPoint

PDF Conversions:

All **.md** files in **docs/** have been automatically compiled into formatted **.pdf** files located in the **docs/** directory for printing, grading, and submission.